

Paso 13 - Todo junto | Ejercicios

Estos ejercicios se hacen **sobre tu calculadora de notas terminada** (el paso 13 de la guía), junto con tus clases `Notas` y `Validador`. Los tres archivos tienen que estar en la misma carpeta, y se compilan juntos:

```
javac Notas.java Validador.java Paso13Notas.java
java Paso13Notas
```

Cada ejercicio la deja mejor que antes, y el orden importa.

► **Plantillas:** [plantillas/](#) — un `.java` por ejercicio, con el enunciado, la salida esperada y los `// TODO` donde va tu código. Si las usas, el archivo ya viene creado (`B13_Calculadora.java`) y **ese** nombre manda sobre el que diga el enunciado.

✓ **Solucionario:** [soluciones/](#) — el código de cada ejercicio, la salida real y por qué se resuelve así. Ábrelo **después** de intentarlo.

Cómo se lee cada ejercicio: el título dice qué hay que lograr; el bloque de abajo es la **salida esperada**, o sea lo que tiene que aparecer en tu pantalla (en los programas que preguntan cosas, lo que va después de la pregunta es lo que **escribe el usuario**); las citas como esta son pistas. Si no sabes por dónde partir, abre la plantilla: los `// TODO` numerados son los pasos, en orden. Compila y ejecuta después de cada uno.

B13.1 - El alumno que aprobó y reprobó a la vez

Qué practicas: detectar un error que no es de sintaxis sino de diseño, y decidir en qué método corregirlo.

Ingresa un alumno con dos notas: `3.9` y `4.0`. El programa muestra:

```
Promedio: 4.0
REPROBADO, le faltan 0.0
```

1. Explica en un comentario por qué pasa. (El promedio real es 3.95.)
2. Arréglalo agregando a `Notas`:

```
public static double redondear(double valor)    // 3.95 -> 4.0
```

Pista: `Math.round(valor * 10) / 10.0`. Piensa por qué lleva el `10.0` y no el `10`.

3. Úsalo en `promedio`, o en `informe`. **Decide dónde va y justifícalo:** ¿el redondeo es parte de calcular el promedio, o solo de mostrarlo?

Este es el ejercicio más importante de la hoja: el programa no tenía ningún error, hacía exactamente lo que decía el código.

B13.2 - El informe completo

Qué practicas: enriquecer un programa reutilizando métodos de tu clase `Notas`, sin escribir ningún ciclo nuevo.

Agrega al recuadro de cada alumno la nota más alta, la más baja y cuántas rojas. `maxima` y `contarRojas` ya existen en `Notas`, y `minima` la escribiste en el B10.1: **no escribas ningún `for` nuevo** en `Paso13Notas`, solo llamadas a `Notas`. (Si no hiciste el B10.1, escribe `minima` ahora: son cuatro líneas.)

```
===== Ana Perez =====
Nota 1: 5.5
Nota 2: 6.2
Nota 3: 3.1
Promedio: 4.9
Maxima: 6.2   Minima: 3.1   Rojas: 1
APROBADO
```

B13.3 - El resumen del curso

Qué practicas: declarar acumuladores antes de un ciclo para que sobrevivan entre vueltas y sirvan para un resumen final.

Al salir del programa, muestra un resumen de todos los alumnos que se ingresaron:

```
Otro alumno? (s/n): n

===== RESUMEN DEL CURSO =====
Alumnos ingresados: 3
Aprobados: 2 (66.7%)
Reprobados: 1
Mejor promedio: 5.4 (Ana Perez)
```

Necesitas acumuladores **antes** del `do-while`, que crezcan en cada vuelta: un contador de alumnos, uno de aprobados, y dos variables para el mejor promedio y su dueño. Para el mejor, el truco del paso 8: parte suponiendo que el primero es el mejor.

B13.4 - El ramo con ponderaciones

Qué practicas: decidir en tiempo de ejecución cuál versión sobrecargada de un método llamar, según lo que responde el usuario.

Después de pedir el nombre, pregunta si el promedio es simple o ponderado. Si es ponderado, son **tres** notas que valen 30%, 30% y 40%, y el promedio se calcula con la sobrecarga del ejercicio B12.3.

```
Nombre del alumno: Luis Soto
Simple o ponderado? (s/p): p
Nota 1 (30%): 5.0
Nota 2 (30%): 6.0
Nota 3 (40%): 4.0
    Promedio ponderado: 4.9
    APROBADO
```

Usa `leerSiNo` o un `leerTexto` validado para la pregunta.

B13.5 - Desafío final: el curso completo

Qué practicas: mantener dos arreglos sincronizados a mano, ordenarlos con selección, y notar por qué ese diseño se vuelve frágil.

Guarda **todos** los alumnos, no solo el que está en pantalla:

- Un `String[] nombres` y un `double[] promedios` (o dos `ArrayList`, del paso 8).
- Al salir, la lista completa ordenada de mayor a menor promedio.

```
===== NOTAS FINALES =====
1. Ana Perez      5.4  APROBADO
2. Luis Soto      4.9  APROBADO
3. Pedro Diaz     3.7  REPROBADO
```

Para ordenar: el método más simple es recorrer el arreglo buscando el mayor, imprimirlo, marcarlo como usado y repetir. Hay formas mejores, pero esa la puedes escribir hoy con lo que ya sabes.

Pregunta de cierre: ¿no sería más cómodo que el nombre y las notas de un alumno viajaran **juntos**, en vez de en dos arreglos paralelos que hay que mantener sincronizados a mano?

Esa incomodidad tiene nombre y es la razón de ser de todo lo que viene: se llama **objeto**, y empieza en la [demo 1](#).

Con esto terminaste `00-base`. Lo que sigue:

1. [EJERCICIOS-00-ARCHIVOS.md](#): 15 minutos sobre la regla "una clase, un archivo".
2. **Demo 1**: clases, objetos y `new`.